

Subagent Invocation & Context Passing

การเรียก subagent และการส่ง context อย่างถูกต้อง

Domain 1 · Task 1.3 — Claude Certified Architect

Subagent Invocation คืออะไร?

- คือกลไกที่ **coordinator** ใช้ "สั่งเกิด" (spawn) subagent ขึ้นมาทำงานย่อย ผ่าน `Task` tool (ใน Claude Code v2.1.63 ถูกเปลี่ยนชื่อเป็น `Agent`)

⚠ Hard requirement

`allowedTools` ของ coordinator **ต้องมี** `"Task"` (หรือ alias `"Agent"`) อยู่ด้วย

ถ้าไม่ตั้งค่านี้ → เรียก subagent **ไม่ได้เลย**

จุดที่ข้อสอบชอบถาม: **"ทำไม spawn ไม่สำเร็จ"**

AgentDefinition: นิยาม subagent แต่ละตัว

subagent แต่ละตัวต้องมี AgentDefinition ที่ระบุ 3 องค์ประกอบ

description

บอกว่า agent นี้ทำอะไร

system prompt

กำหนดบทบาทและพฤติกรรม

tool restrictions

จำกัดเครื่องมือให้แคบตามหน้าที่



หลักการ

scope tools ให้พอดีกับ role — **ไม่ให้ subagent มีเครื่องมือเกินความจำเป็น**

หลักการ Context Isolation

★ ห้ามพลาด

subagent ไม่ inherit ประวัติ

subagent **ไม่สืบทอด** ประวัติสนทนาของ coordinator โดยอัตโนมัติ — ทุกครั้งที่เรียกคือ**การเรียกอิสระ**

subagent เข้าไม่ถึง:

- system prompt ของ coordinator
- ข้อความรอบก่อน
- ผลลัพธ์ของ subagent ตัวอื่น

🔑 Key Concept

อย่าสมมติว่า subagent ถึงผลงานของ agent ก่อนหน้าเองได้

ทุกครั้งที่ subagent ต้องใช้ coordinator **ต้องใส่ลงไป**
prompt อย่างชัดเจน

(explicit context passing)

3 กฎการส่ง Context

- 1 **ส่งผลลัพธ์ครบถ้วน** — ส่ง findings ทั้งหมดจาก agent รอบก่อนไปให้ agent ปลายทาง อย่าคาดหวังว่าปลายทางจะไปดึงผลลัพธ์กลับมาเองได้
- 2 **ใช้ structured format** แยก content ออกจาก metadata
แบบ source URL, document name, page number และ confidence level ไปพร้อมทุก claim
- 3 **ระบุ "เป้าหมาย" ไม่ใช่ "ขั้นตอน"** — prompt ควรบอก goal ให้ subagent ปรับตัวเอง ไม่ใช่ไล่สิ่งทีละ step

Parallel vs Sequential

★ ออกสอบบ่อย

✓ Parallel spawning

subagent ที่**อิสระต่อกัน** (ไม่ต้องรอผลของกันและกัน)
ควรถูก spawn พร้อมกัน

coordinator emit หลาย Task tool calls ภายในการตอบครั้งเดียว → **low latency ได้มาก**

กับटक

การเสนอให้เรียกแบบ **sequential** ทั้งที่งานรันขนานกันได้ = คำตอบผิดที่ข้อสอบวางไว้ล่อ

ต้องแยกให้ออก:

independent → ขนานได้

dependent → ต้องเรียงลำดับ

fork_session vs --resume

fork_session

สร้าง **branch อิสระ** จาก baseline ร่วมกัน เพื่อแตกไปสำรวจหลายทิศทาง

(divergent exploration จากจุดตั้งต้นเดียวกัน)

--resume

ทำงานต่อจาก **thread เดิม** ที่เจาะจงไว้ (สาน context เดิมต่อ)



Key Concept

fork = แยกกิ่งจากฐานร่วมเพื่อลองหลายเส้นทาง

resume = เดินต่อในเส้นทางเดิม

⚡ ข้อสอบชอบสลับสองคำนี้ให้สับสน

Attribution Failure Pattern

★ ออกสอบบ่อย

อาการ

synthesis agent ผลิตรายงานที่หลาย claim **ไม่มีแหล่งอ้างอิง** (missing citations)

ทั้งที่ web search subagent (คืน URL, title, snippet) และ document analysis subagent (คืน page reference) ทำงานถูกต้องทุกตัว

🔍 หลักวินิจฉัย

เมื่อ citation หาย **อย่าโทษ synthesis agent** — ให้ไปดูว่า coordinator **strip metadata** ที่ก่อนส่งต่อหรือไม่
ต้นเหตุมักอยู่ที่ context passing ที่ไม่มี metadata แทบทุกครั้ง



Case Study: รายงานที่อ้างอิงไม่ครบ

อาการ

synthesis agent สร้างรายงานที่ **หลายข้อความอ้างอิงไม่มี source attribution**

สาเหตุ

coordinator ส่ง content ให้ synthesis agent **โดยไม่แนบ structured metadata**

→ source URL, document name, page number ถูก **ตัดทิ้ง** ก่อน handoff

✓ วิธีแก้

บังคับให้ coordinator ส่ง **structured output ควบพร้อม metadata คงสภาพ** (source URL + document name + page number ต่อทุก finding)

ไม่ใช่ไปแก้ system prompt ของ synthesis agent

! Exam Traps ที่ต้องระวัง

ตัวหลอก #1 — สมมติว่า subagent เข้าถึง context ของตัวอื่นได้เอง

× จริง ๆ context แยกกันสิ้นเชิง — coordinator ต้องส่งให้ชัดเจน

ตัวหลอก #2 — โทษ synthesis agent เรื่อง citation หาย

× ต้นเหตุคือ context passing ที่ไม่มี metadata

ตัวหลอก #3 — เรียก subagent แบบ sequential ทั้งที่รันขนานได้

× เสีย latency ฟรี

ตัวหลอก #4 — สับสน `fork_session` กับ `--resume`

× fork = แยกกิ่ง · resume = เดินต่อเส้นเดิม



Practice Scenario

ลองคิดก่อนดูเฉลย

โจทย์: synthesis agent ผลิตรายงานที่หลาย claim ไม่มี source attribution ทั้งที่ web search subagent (คืน URL, title, snippet) และ document analysis subagent (คืน page reference) ได้รับการตรวจสอบแล้วว่าทำงานถูกต้อง — **ปัญหาที่แท้จริงอยู่ที่ไหน?**

A. ควรให้ synthesis agent เข้าถึง web search tool ได้โดยตรง

B. coordinator ส่ง content ต่อโดยไม่มี structured metadata (source URL, document name, page number)

C. system prompt ของ synthesis agent ขาดคำสั่งเรื่อง citation

D. web search subagent ต้องเปลี่ยน result format ใหม่

 **หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อดูเฉลย**



เฉลย Practice Scenario

คำตอบที่ถูก — Option B

coordinator ส่ง content ต่อโดย**ตัด structured metadata ทิ้ง** ทำให้ปลายทางไม่มี source URL / document name / page number จะอ้างอิง

→ ต้นเหตุคือ **context passing** ไม่ใช่คุณภาพของ subagent

A ผิด — ผิดหลัก context isolation; subagent ต้นทางทำงานถูกต้องอยู่แล้ว การเพิ่ม tool ไม่ได้แก้ metadata ที่หายระหว่างทาง

C ผิด — แก้ผิดจุด; ต่อให้สั่งให้ cite ก็ cite ไม่ได้เมื่อ metadata ไม่ถูกส่งมา

D ผิด — subagent ค้นผลถูกต้องครบแล้ว ปัญหาเกิดตอน coordinator ส่งต่อ ไม่ใช่ที่ format ต้นทาง



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

synthesis agent ผลิตรายงานที่หลาย claim ไม่มี source attribution ทั้งที่ web search subagent (คืน URL, title, snippet) และ document analysis subagent (คืน page reference) ถูกยืนยันแล้วว่าทำงานถูกต้องทั้งคู่ — **root cause** ที่น่าจะเป็นไปได้ที่สุดคืออะไร?

- A. system prompt ของ synthesis agent ไม่มีคำสั่งให้ cite แหล่งอ้างอิง
- B. coordinator ส่ง content ให้ synthesis agent โดยไม่มี structured metadata — source URL, document name และ page number ไม่ถูกแนบไปด้วย
- C. web search subagent ต้องคืนผลในรูปแบบอื่นที่ synthesis agent parse ได้
- D. ควรให้ synthesis agent เข้าถึง web search tool ได้โดยตรงเพื่อตรวจสอบแหล่งเอง



เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option B

context passing ต้องส่ง structured data ที่แยก content ออกจาก metadata ถ้าไม่มี source URL และ document name ในข้อมูลที่ส่งให้ synthesis agent มันก็ไม่มีข้อมูล attribution ให้ใช้ ไม่ว่า instruction จะดีแค่ไหน

ทำไมข้ออื่นผิด

- A — ต่อให้มีคำสั่งให้ cite ก็ cite สิ่งที่ไม่เคยได้รับไม่ได้ ถ้า coordinator strip metadata ทิ้งก่อน ไม่มี prompt ไหนรู้ข้อมูลที่หายกลับมาได้
- C — web search subagent ค้นผลที่มีโครงสร้างดีอยู่แล้ว ปัญหาไม่ใช่ format แต่เป็นที่ coordinator ไม่ส่ง metadata ผ่านไปยัง synthesis agent
- D — การให้ web search tool กับ synthesis agent ผิดหลัก scoped tool access และทำลาย hub-and-spoke architecture ทางแก้คือ context passing ที่ถูกต้อง ไม่ใช่ให้ tool เกิน role



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

coordinator ต้อง spawn web search subagent และ document analysis subagent สำหรับงานวิจัยที่อิสระต่อกัน — **วิธี spawn ที่มีประสิทธิภาพที่สุดคืออะไร?**

- A. เรียก web search subagent ก่อน รอผล แล้วค่อยเรียก document analysis subagent
- B. emit หลาย Task tool calls ในการตอบครั้งเดียวของ coordinator เพื่อ spawn ทั้งสอง subagent แบบ parallel
- C. สร้าง shared message queue ที่ทั้งสอง subagent อ่านพร้อมกันได้
- D. เรียกทั้งสอง subagent ด้วย prompt เหมือนกันเพื่อ cross-verify ผลของกันและกัน

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option B

การ emit หลาย Task tool calls ในการตอบครั้งเดียวจะ spawn subagent อิสระพร้อมกัน ลด latency เมื่อเทียบกับการเรียกแบบ sequential ที่กระจายข้ามหลาย turn

ทำไมข้ออื่นผิด

- A — การเรียกแบบ sequential เพิ่ม latency โดยไม่จำเป็นเมื่องานอิสระต่อกัน ไม่มี subagent ตัวไหนต้องรอผลของอีกตัวก่อนเริ่มงาน
- C — shared message queue ทำลาย hub-and-spoke architecture การสื่อสารทั้งหมดต้องผ่าน coordinator ไม่ใช่ผ่าน infrastructure ที่ใช้ร่วมกัน
- D — prompt เหมือนกันเปลี่ยนทรัพยากรและไม่แก้ปัญหาลatency แต่ละ subagent ควรได้ prompt ที่ปรับตาม role และ subtopic ของตัวเอง



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

allowedTools ของ coordinator มี ["web_search", "read_file"] แต่ **ไม่มี** "Task" — จะเกิดอะไรขึ้นเมื่อ **coordinator พยายาม spawn subagent?**

- A. coordinator spawn subagent ได้ แต่ความสามารถลดลง
- B. coordinator spawn subagent ไม่ได้เลย — Task ต้องอยู่ใน allowedTools
- C. coordinator fallback ไปเรียก tool ของ subagent โดยตรงอัตโนมัติ
- D. coordinator spawn subagent ได้ แต่ subagent inherit tool ของ coordinator แทน tool ของตัวเอง

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

Task คือ hard gate ของการ spawn subagent ถ้าไม่มี Task ใน allowedTools coordinator ก็ไม่สามารถเรียก subagent ตัวใดได้เลยในเชิงกายภาพ ไม่มี fallback หรือ workaround ใด ๆ

ทำไมข้ออื่นผิด

- A — เมื่อไม่มี Task จะไม่มี subagent ถูก spawn เลย ไม่ใช่หมดความสามารถลง แต่คือ "เรียกไม่ได้เลย" โดยสิ้นเชิง
- C — ไม่มี fallback อัตโนมัติ coordinator ใช้ได้แค่ tool ใน allowedTools เมื่อไม่มี Task ก็ไม่มีกลไก invoke subagent
- D — ไม่มี subagent ถูกสร้างขึ้น Task จำเป็นต่อการ spawn เรื่อง tool inheritance ไม่เกี่ยวเมื่อการ spawn เองยังทำไม่ได้



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

developer ใช้ fork_session หลังวิเคราะห์ codebase เพื่อลอง refactoring strategy สองแบบ — ข้อความใดอธิบายพฤติกรรมของ fork ได้ถูกต้อง?

- A. ทั้งสอง fork ใช้ memory ร่วมกันและเห็นผลของกันและกันแบบ real time
- B. แต่ละ fork ทำงานอิสระหลังจุด branching — การเปลี่ยนแปลงใน fork หนึ่งไม่กระทบอีก fork
- C. fork ที่สองได้รับ summary ของ finding จาก fork แรกอัตโนมัติ
- D. fork เหมือนกับ --resume session ทุกประการและใช้แทนกันได้

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option B

fork_session สร้าง branch อีกระยะจาก baseline การวิเคราะห์ร่วมกัน หลัง fork แต่ละ branch ทำงานอิสระ ไม่เห็นผลของกันและกัน และการเปลี่ยนใน branch หนึ่งไม่กระทบอีก branch

ทำไมข้ออื่นผิด

A — fork ถูก isolate อย่างชัดเจน แชนจ์ context เฉพาะ baseline ถึงจุด branching แต่หลังจากนั้นอิสระสิ้นเชิง ไม่มี shared memory

C — fork อิสระต่อกัน fork ที่สองไม่รับรู้กิจกรรมของ fork แรกหลังจุด branching

D — fork_session กับ --resume ทำหน้าที่ต่างกันสิ้นเชิง fork แรกเพื่อสำรวจทางเลือก ส่วน resume สานสนทนาเดิมต่อ



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

เมื่อออกแบบ coordinator prompt สำหรับ subagent — **แนวทางใดทำให้ subagent ทำงานได้ดีกว่า?**

A. ให้คำสั่งแบบ procedural ทีละ step อย่างละเอียดว่า subagent ต้องทำงานอย่างไร

B. ระบุ research goal และ quality criteria ปล่อยให้ subagent ปรับแนวทางเอง

C. ใส่ conversation history ทั้งหมดของ coordinator เพื่อให้ subagent มี context มากที่สุด

D. ทำ prompt ให้สั้นที่สุดเพื่อไม่ให้ subagent รับข้อมูลล้นเกิน

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option B

prompt แบบ goal-oriented ทำให้ subagent ปรับตัวได้ การระบุว่าจะบรรลุอะไรและ quality criteria อะไร ช่วยให้ subagent ปรับแนวทางเมื่อเจอสถานการณ์ที่ไม่คาดคิด

ทำไมข้ออื่นผิด

A — คำสั่ง procedural บังคับ subagent ไว้และกันไม่ให้ปรับแนวทาง ถ้าเจอสถานการณ์ไม่คาดคิด ขั้นตอนที่แข็งทื่ออาจให้ผลแย่

C — subagent มี context แยกโดยดีไซน์ การใส่ conversation history ทั้งหมดเปลืองและอาจทำให้สับสนด้วยข้อมูลที่ไม่เกี่ยวข้อง ส่งเฉพาะ context ที่จำเป็นต่องาน

D — prompt สั้นเกินไปอาจขาด context ที่จำเป็นต่อคุณภาพผลงาน prompt ควรระบุ goal เฉพาะ, quality criteria, finding ที่เกี่ยวข้องกับ agent ก่อนหน้า และ output format ที่คาดหวัง